CS4421: Computer Organisation Assessment

Oct 12, 2026 · Mark Burkley

Block 1, Year 1

Code submission date: 29th October 2026

Interview date: 30th October 2026

13.3% of Block

Assessment description

In this assessment your team writes a Java program that reads information about the computer it runs on and reports it in a clear text form: processor, memory, PCI devices and USB devices.

The data comes from files that Linux provides under `/proc` and `/sys`, which the "Where to find the data" section describes. You will be provided with Java classes `cpuInfo`, `memInfo`, `pciInfo` and `usbInfo` that read those files and return the values. You can use them as they are, extend them, or write your own readers. Your marks depend on what you do with the information and how well you can explain it.

Your task is to find out what your machine reports, store it, analyse it, and present it so that a reader can understand the capabilities of the machine. An example of the kind of sentence your output might contain: "this machine has 4 cores and 16 GiB of RAM, of which 8 GiB is currently in use." You will be asked in the interview to explain what the numbers mean and where they came from.

Core requirements (every team)

Your program must do all of the following and print the results as readable text.

1. **Processor.** Report the number of CPU cores and the processor model. Measure the load by reading the busy and idle counts twice, about one second apart, and working out the ratio from the difference. Report the load per core and the average across all cores.
2. **Memory.** Report total memory and used memory, and the percentage in use.
3. **PCI devices.** List every PCI device with its bus, device and function numbers, vendor ID, device ID and class. Store them in an array or list.
4. **USB devices.** List every USB device with its vendor ID and product ID, stored in an array or list. A virtual machine or the cloud server may have none; your program must still run and say so.
5. **Names.** Look up each vendor and device ID in the `pci.ids` and `usb.ids` files provided, so that vendor `0x8086` appears as "Intel Corporation" and so on. Show the raw ID when no name is found.
6. **Analysis.** Sort the PCI list in two ways, alphabetically by vendor name and numerically by bus, device and function. Calculate at least one summary, such as the number of devices per vendor or per class.
7. **Output.** Present everything as text, and make it easy to read.

The core must run from a terminal with no graphical display, so that it works on the alderaan cloud server as well.

Extension (choose at least one)

The extension is where your team shows creativity and relates the project to the lectures. Choose one of these suggestions or come up with your own.

- **Memory hierarchy explorer.** Write a Java loop that reads a large array with different strides and different working-set sizes, and time it. Plot or tabulate time per access, and use the result to estimate the cache line size and the cache sizes. Compare your estimates with what the machine reports under `/sys/devices/system/cpu/cpu0/cache/`.
- **Load and parallelism.** Run a processor-heavy task on 1, 2, 4 and more threads and measure the speedup. Show the per-core load while it runs, and compare your measured speedup with the limit predicted by Amdahl's law.
- **Presentation.** Export your results as an HTML page, SVG chart or CSV file that you can open on your own computer. The text version must still work on its own.
- **Your own idea.** For example, compare the PCI lists of two different machines, or explain why a virtual machine shows the devices it does.

Whichever you choose, be ready to explain in the interview what your measurements show and why.

Where to find the data

Linux presents hardware information as ordinary text files. You can read them in a terminal with `cat` before you write any code, and then read them from Java as you would any other file.

Information	File or folder	What to read
Processor model	`/proc/cpuinfo`	The `model name` line (one block per core)
Number of cores	`/proc/cpuinfo`	Count the `processor` lines
CPU busy and idle time	`/proc/stat`	Lines starting `cpu0`, `cpu1` and so on; the numbers are time spent in each state since boot, so read twice and subtract
Memory	`/proc/meminfo`	`MemTotal` and `MemAvailable`, in kB
PCI devices	`/sys/bus/pci/devices/`	One folder per device, named like `0000:00:02.0` (bus `00`, device `02`, function `0`); files `vendor`, `device` and `class` hold hexadecimal IDs

Information	File or folder	What to read
USB devices	`/sys/bus/usb/devices/`	Files `idVendor` and `idProduct` in each device folder; may not exist on a virtual machine
Cache sizes	`/sys/devices/system/cpu/cpu0/cache/`	Folders `index0`, `index1` and so on, each with `level`, `type`, `size` and `coherency_line_size`
Vendor and device names	`pci.ids`, `usb.ids`	Text files provided with the assignment: lines for vendors, indented lines for devices

Two cautions. On an arm64 machine `/proc/cpuinfo` has no `model name` line, so report the CPU implementer and part numbers or say the model is not available. Some virtual machines leave out the cache folder, so your program must cope with a missing file and not crash.

The provided `cpuInfo`, `memInfo`, `pciInfo` and `usbInfo` classes already read these files. Read their code before you use them, because you may be asked how they work. You may modify them or replace them with your own.

Submission guidelines

You will work in teams to design and write the Java code for this assignment. Use GitHub as your version control tool, so that every team member's contributions are visible. Your repository must include a short README that says how to build and run the program, and which file each piece of information comes from.

Assessment evaluation: team interview

The interview lasts about 15 minutes with members of ISE staff. You will be asked questions about the code you produce and the choices you made. Once you have completed your interview you may leave. The schedule for team interviews will be published separately.

Your grade is based mainly on your ability to answer questions about your submission, so every member of the team must be able to explain any part of it. Expect questions such as: what a PCI bus, device and function number identify; what a vendor ID is and why a lookup file is needed; how you calculated CPU load from two readings; and what your extension measured and why.

Marking scheme

The marking scheme is as follows. The marks come from your answers in the interview.

#	Item	Description	Marks
1	Purpose	Meets brief and subject knowledge	20
2	Execution and integration	Can discuss functionality	20
3	Execution and integration	Can discuss design choices	10
4	Execution and integration	Programming skills; can discuss how this submission relates to topics covered in class	10
5	Creativity and communication	Communication	5
6	Creativity and communication	Problem solving and critical thinking	5
7	Creativity and communication	Creativity and innovation (including your extension)	30
		Total	100

Slight adjustments to this brief may be made by the module coordinator at their discretion.

Academic guidelines

Late submissions. Late submission loses 5% for each day late.

Plagiarism. Plagiarism will not be tolerated. Maximum penalties will be awarded (see the UL policy on plagiarism). The declaration below must be included in your submission.

AI and AI-generated content. Students who have used AI or AI-assisted tools must include a statement at the end of their submission, entitled "Declaration of Generative AI and AI-assisted technologies". In that statement, specify the tool that was used and the reason for using it. Overuse or abuse of these tools will be penalised. Students must record all prompts and responses generated and save them for the full academic year. Teaching staff may ask for these at any point in the year to consider the extent of any AI-generated content.

Plagiarism declaration: ISE 2026

We declare that this material, which we now submit for assessment, is entirely our own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text. We understand that plagiarism, collusion, and copying are grave and serious offenses in the university and accept the penalties that would be imposed should we engage in plagiarism, collusion or copying. We understand that any use of AI or AI-assisted tools must be declared fully. We have read and understood the Assignment Regulations set out in the module documentation. This assignment, or any part of it, has not been previously submitted by us or any other person for assessment on this or any other course of study.

Signed: _____

ID: _____